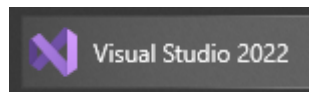


Знакомство со средой программирования Visual Studio 2022 и онлайн платформой Яндекс.Контекст

Упражнение 1. Создание проекта консольного приложения.

1. Создание проекта консольного приложения

Для запуска среды программирования выберите в меню кнопки **Пуск** пункт



. Интерфейс открывшейся страницы представлен на рис. 1.

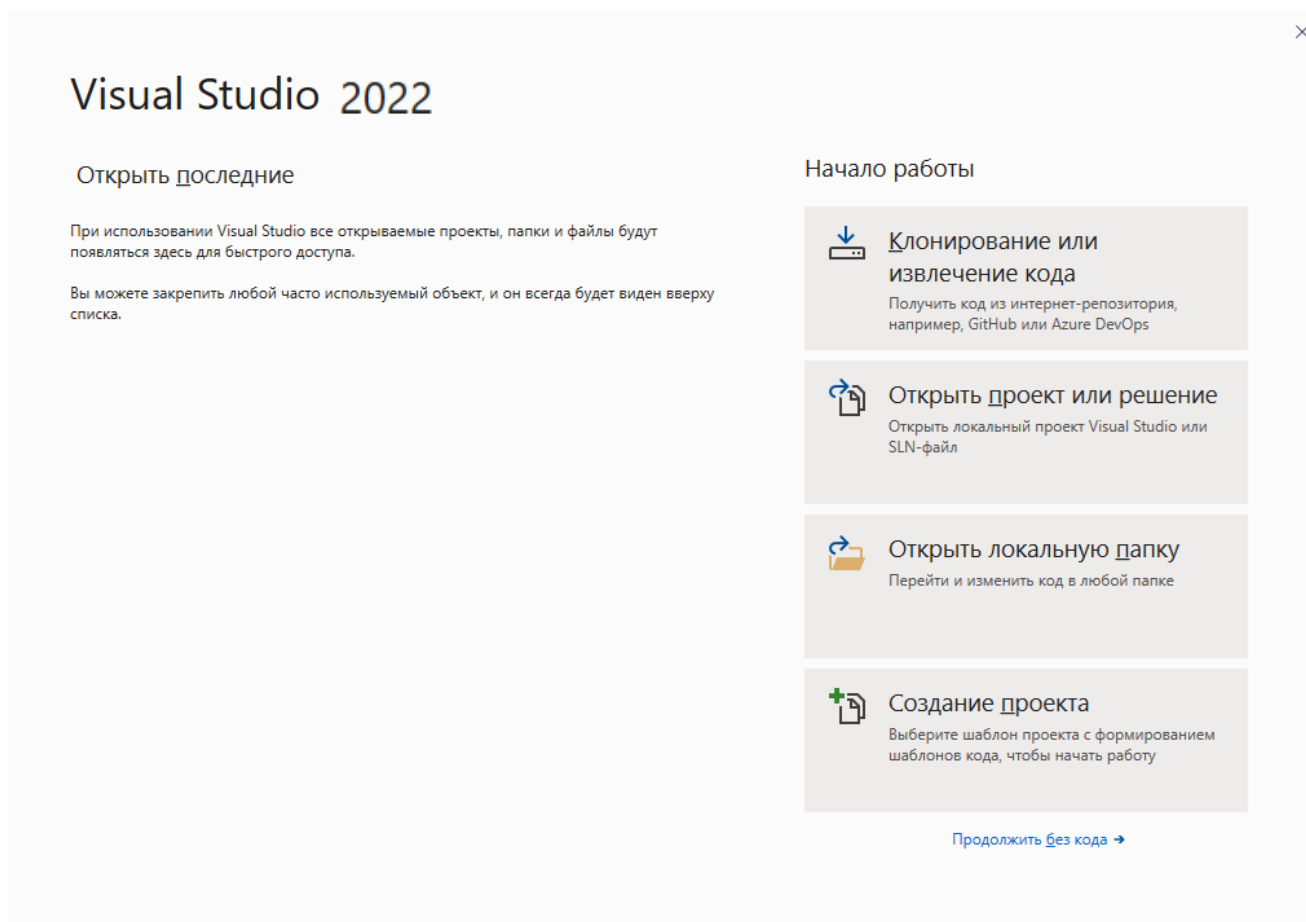


Рис. 1. Начальная страница приложения Visual Studio 2022

Перед написанием программы необходимо создать проект, который будет содержать все файлы разрабатываемого приложения.

После запуска среды программирования для создания проекта необходимо выбрать **Создание проекта** в нижнем левом углу начальной страницы (рис. 2).

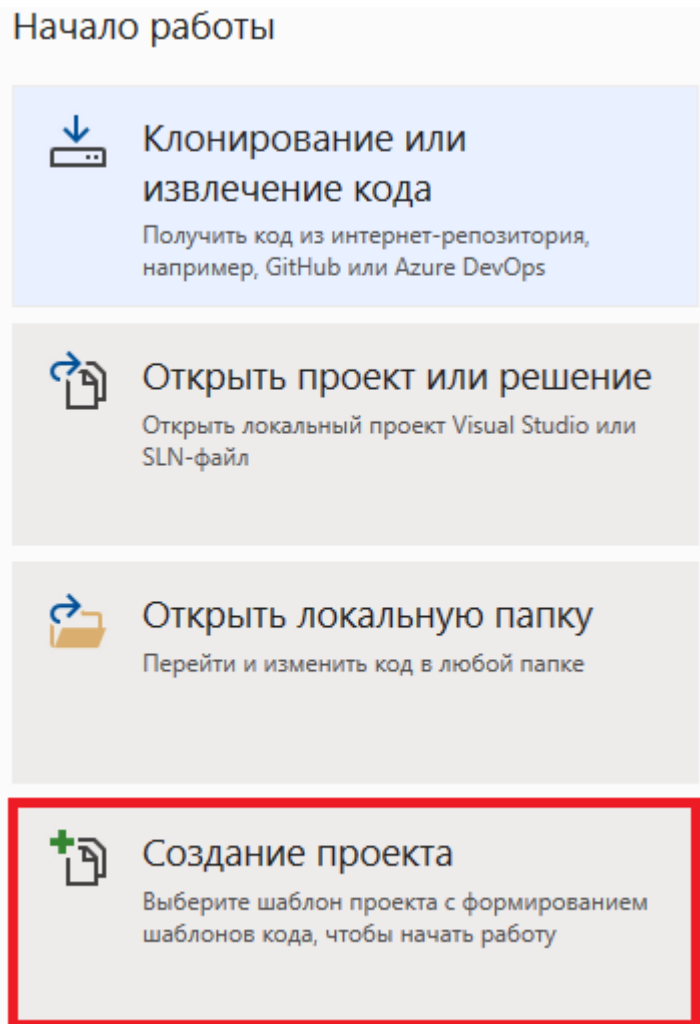


Рис. 2. Создание проекта

В появившемся окне «Создание проекта» (рис. 3) необходимо сначала выбрать язык **C++**, а затем – шаблон создаваемого приложения **Консольное приложение Windows**. Нажать кнопку **Далее**.

В появившемся окне нужно указать с помощью кнопки «...» расположение проекта и ввести его имя в соответствующем поле. Имя решения автоматически дублирует имя проекта (рис. 4).

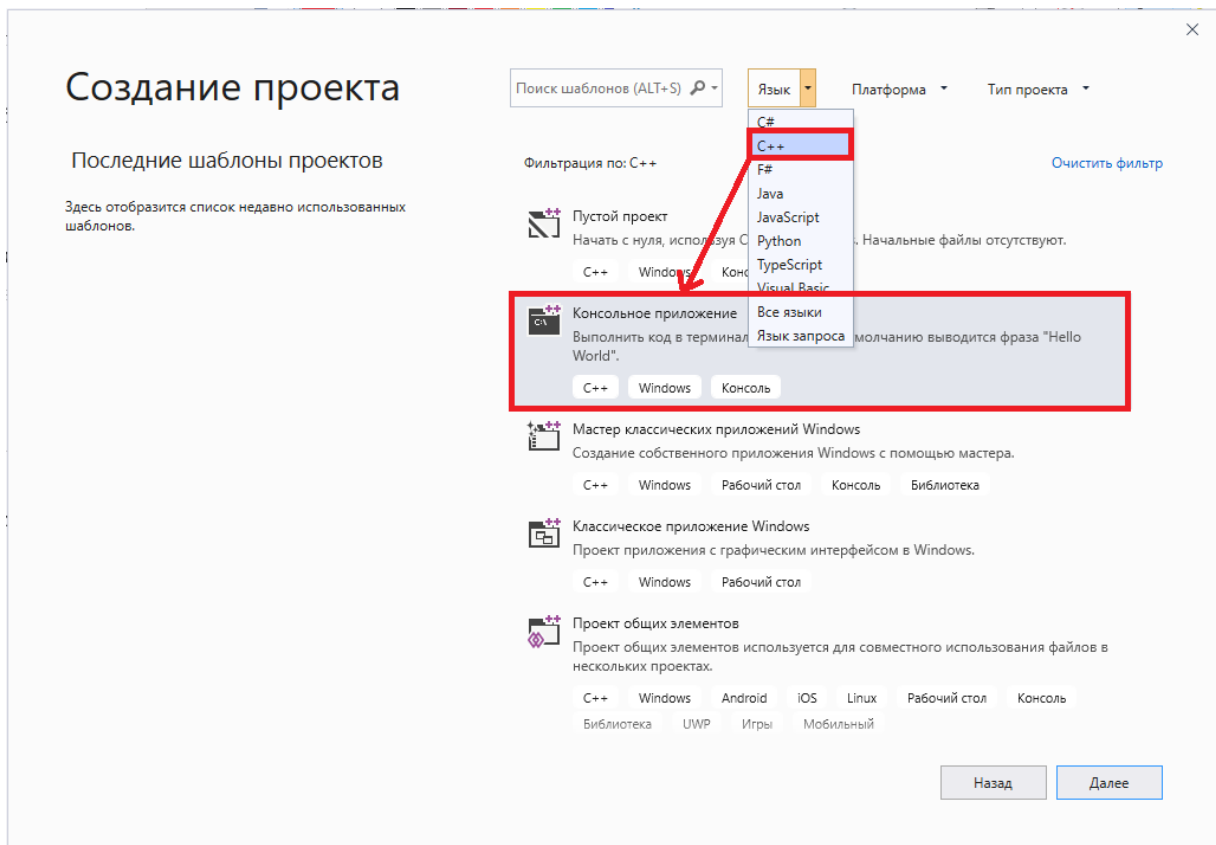


Рис. 3. Создание консольного проекта

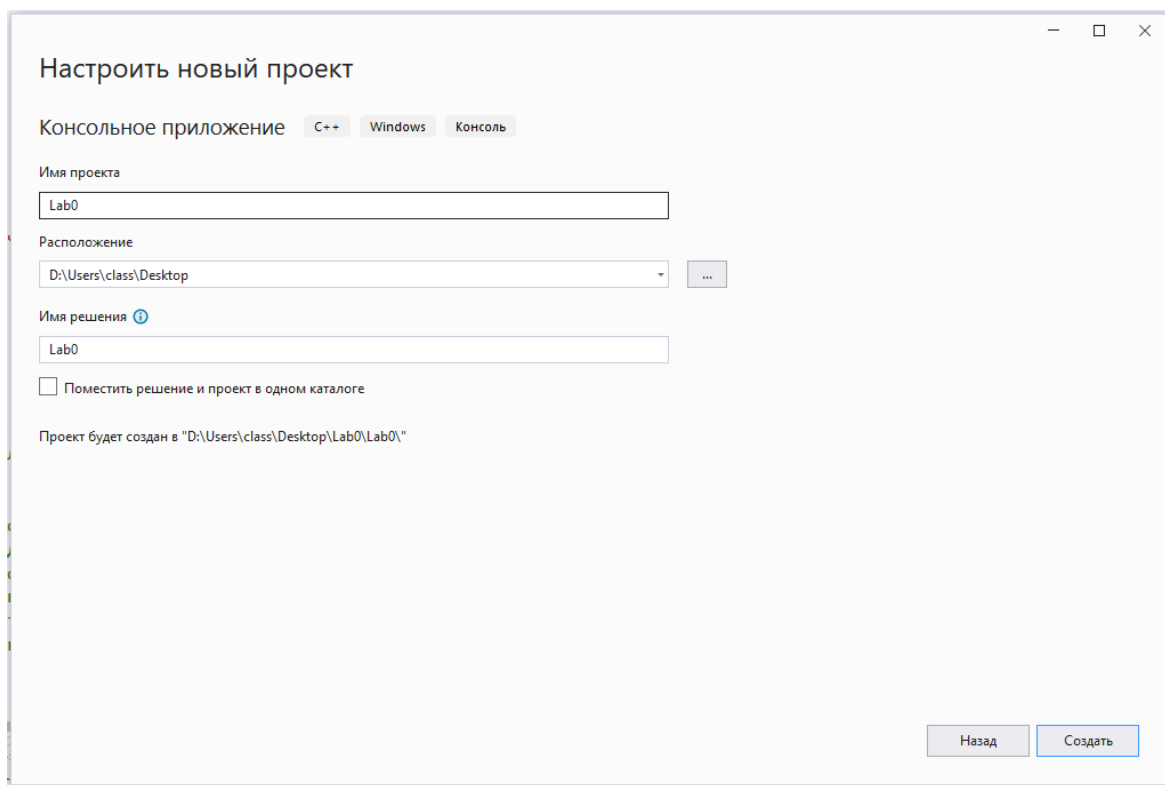


Рис. 4. Создание консольного проекта

Если установлен флажок **Поместить решение и проект в одном каталоге**, то для решения не будет создан отдельный каталог.

После нажатия кнопки **Создать** Visual Studio создаст проект (рис. 5).

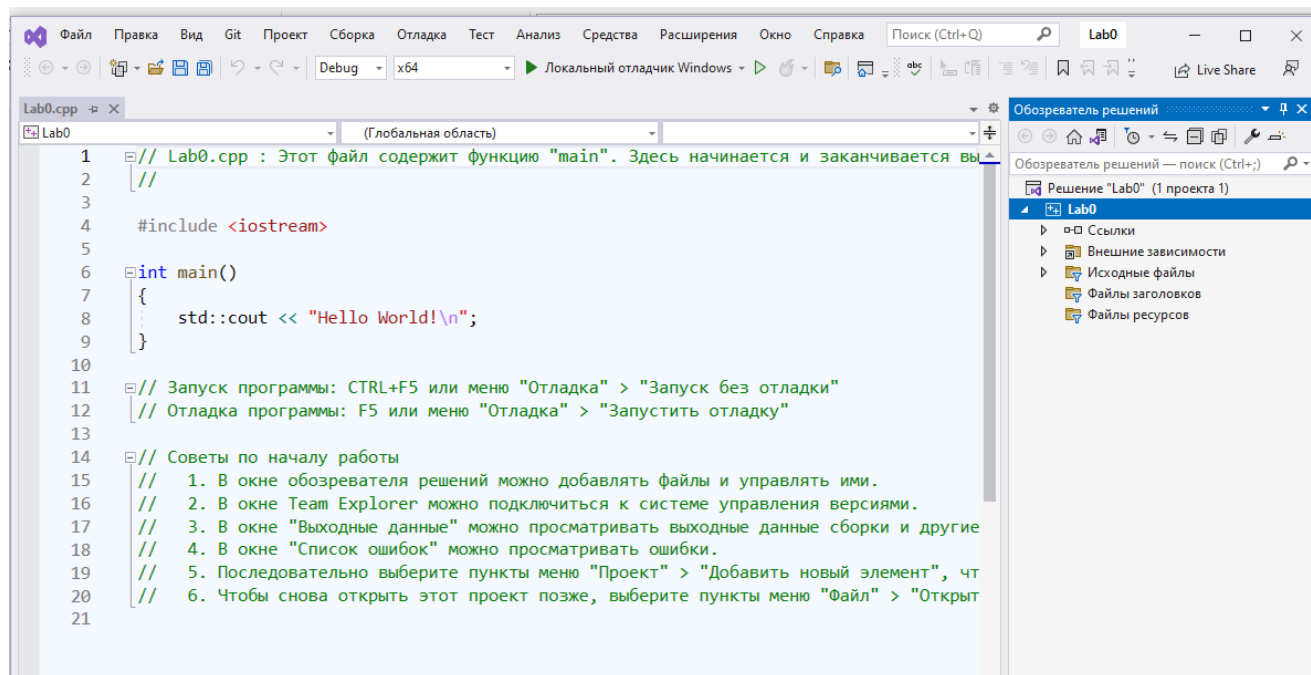


Рис. 5. Созданный проект

При таком способе создания проект содержит только файл программы (в примере **Lab0.cpp**). В более старых версиях Visual Studio в проект также будут включены файлы предкомпилированного заголовка (**pchx.h** в VS2017 и **stdafx.h** в VS2015 и ранее).

Содержимое всех файлов проекта можно просмотреть в окне редактора кода. По умолчанию, при создании нового проекта открывается его главный файл, содержащий функцию **main()** (рис. 5).

2. Файлы проекта консольного приложения

С правой стороны находится панель **Обозреватель решений** (рис. 6).

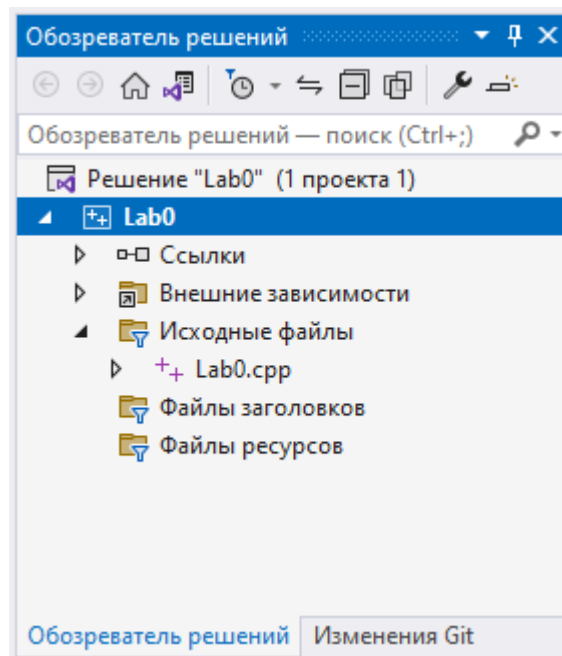


Рис. 6. Окно «Обозреватель решений»

Наиболее важными из созданных файлов являются:

- файл исходного кода **<имя_проекта>.cpp**. В этом файле будет находиться основной код программы, в том числе функция **main()**. Имя этого файла должно совпадать с именем проекта.

- заголовочный (предкомпилированный заголовок) файл **pch.h** или **stdafx.h** в более старых версиях (в этой версии таких файлов нет!).

Предкомпилированные заголовки (англ. pre-compiled headers) – это метод, позволяющий некоторым компиляторам языков программирования C/C++ уменьшить время компиляции, путем предварительной компиляции редко меняющихся файлов исходного кода приложения. То есть данные файлы компилируются один раз и перекомпилируются только в случае изменения одного из них. Таковыми, как правило, являются заголовочные файлы.

В файлах предкомпилированных заголовков обычно размещаются стандартные заголовки, например, наиболее часто используемые **stdio.h**, **stdlib.h**, **string.h** и **math.h**.

3. Настройка среды программирования

В Visual Studio 2022 все окна и панели свободно перемещаются и могут быть либо закрепленными по одной из сторон экрана, либо «висячими». Изменить

положение окна или панели можно путем выделения его заголовка и перетаскиванием мышью. При этом на экране появляется разметка, отображающая возможное размещение данного элемента (рис. 7).

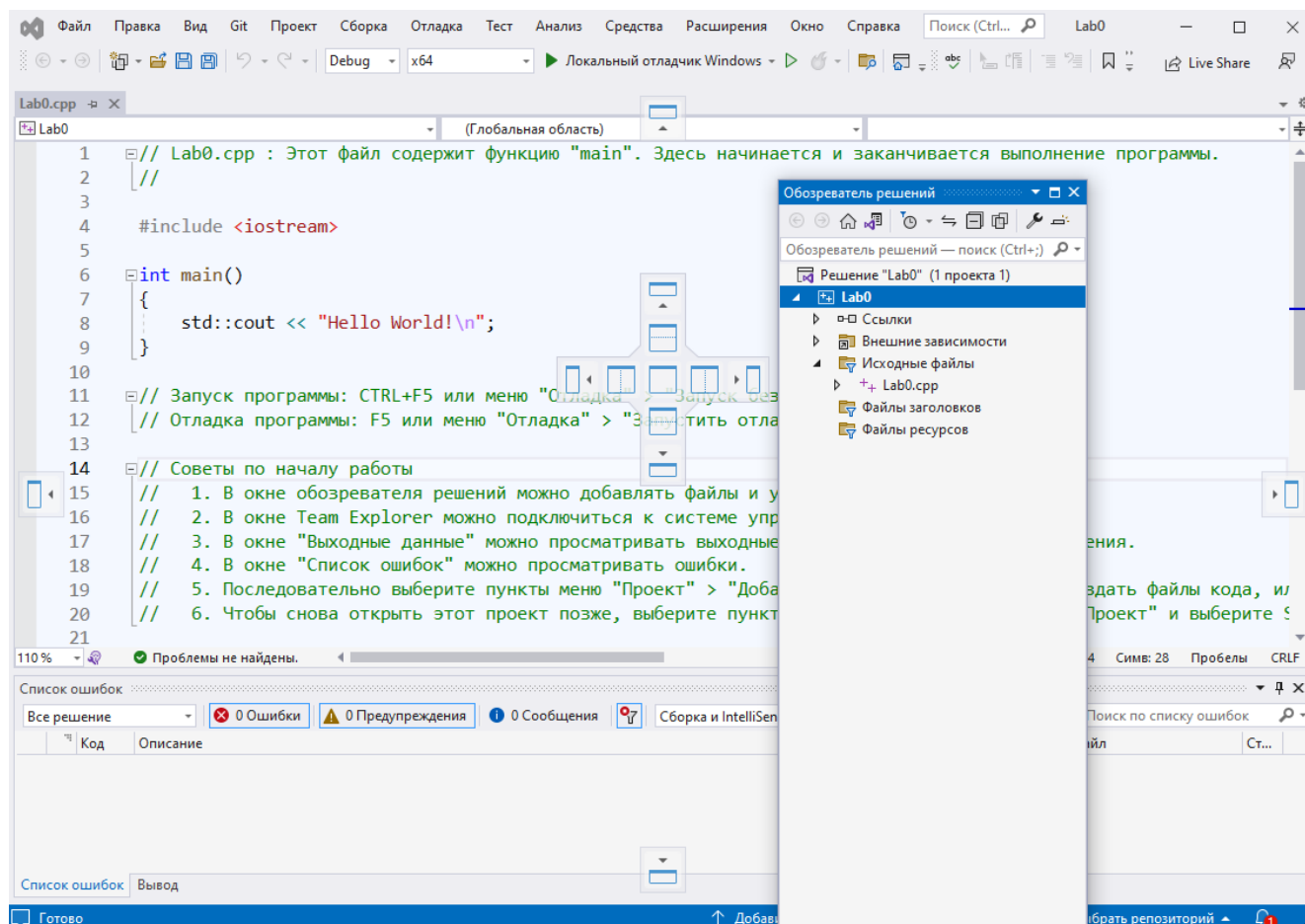


Рис. 7. Перемещение окон

Полный список окон, которые могут отображаться средой программирования, приведен в разделе **Меню – Вид**.

Для удобства отладки программы можно добавить окно **Список ошибок** (**Меню – Вид – Список ошибок**), в котором будут отображаться номера строк кода, содержащие ошибку, и информация о ней.

Упражнение 2. Расположите окна среды программирования наиболее удобным для Вас способом.

4. Запуск и отладка программы

После написания кода программы необходимо выполнить его компиляцию (**Меню – Сборка – Собрать решение** или **Ctrl+Shift+B**). Результаты успешной компиляции отображаются в окне **Вывод** (Рис. 8).

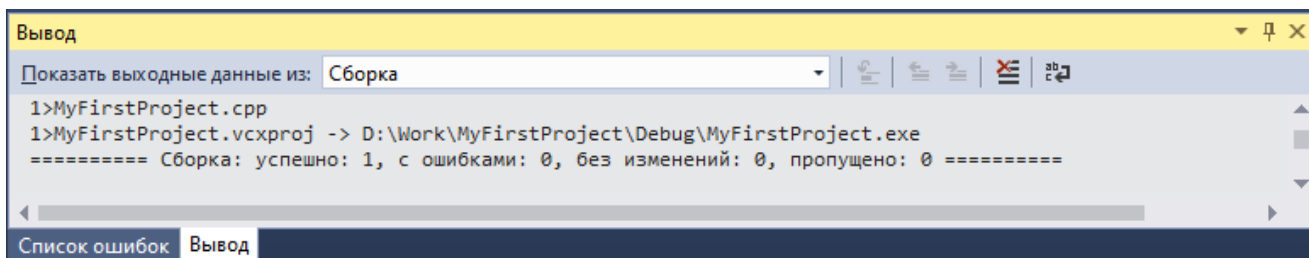


Рис. 8. Сообщение об успешной компиляции

Если исходный код содержит синтаксические ошибки, то информация о них будет приведена в окне **Список ошибок** (Рис. 9).

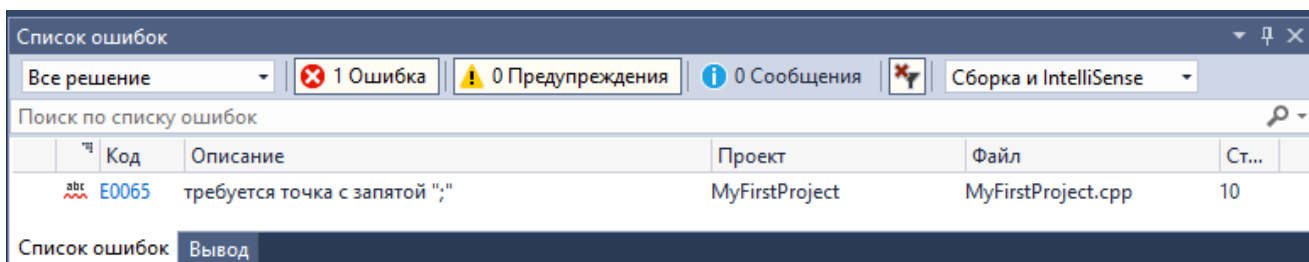


Рис. 9. Сообщение об успешной компиляции

Если компиляция прошла успешно, то проект можно запустить на выполнение: **Меню – Отладка – Начать отладку (F5)** или **Меню – Отладка – Запуск без отладки (Ctrl+F5)**. В первом случае программу можно будет отлаживать (см. пункт 6).

Упражнение 3. Скопируйте приведенный ниже пример в свой проект. Выполните запуск приложения и убедитесь в отсутствии ошибок.

```
#include <stdio>
int main()
{
    int x, y;
    printf("Input x and y: ");
    scanf_s("%d %d", &x, &y);
    printf("%d + %d = %d", x, y, x + y);
    return 0;
}
```

Упражнение 4. Экспериментально определите, какой алгоритм выполняет эта программа. К каждой строке кода напишите комментарий в виде:

```
int x, y; // объявление переменных
```

5. Добавление нового проекта в существующее решение

Часто для удобства несколько проектов, решающих смежные задачи или части одной большой, объединяют в одно решение. Например, удобно все программы к одной лабораторной работе хранить в одном решении.

Для добавления нового проекта в решение необходимо в окне «Обозреватель решений» выбрать самый верхний уровень, щелкнуть правой кнопкой мыши и выбрать **Добавить – Создать проект...** (рис. 10).

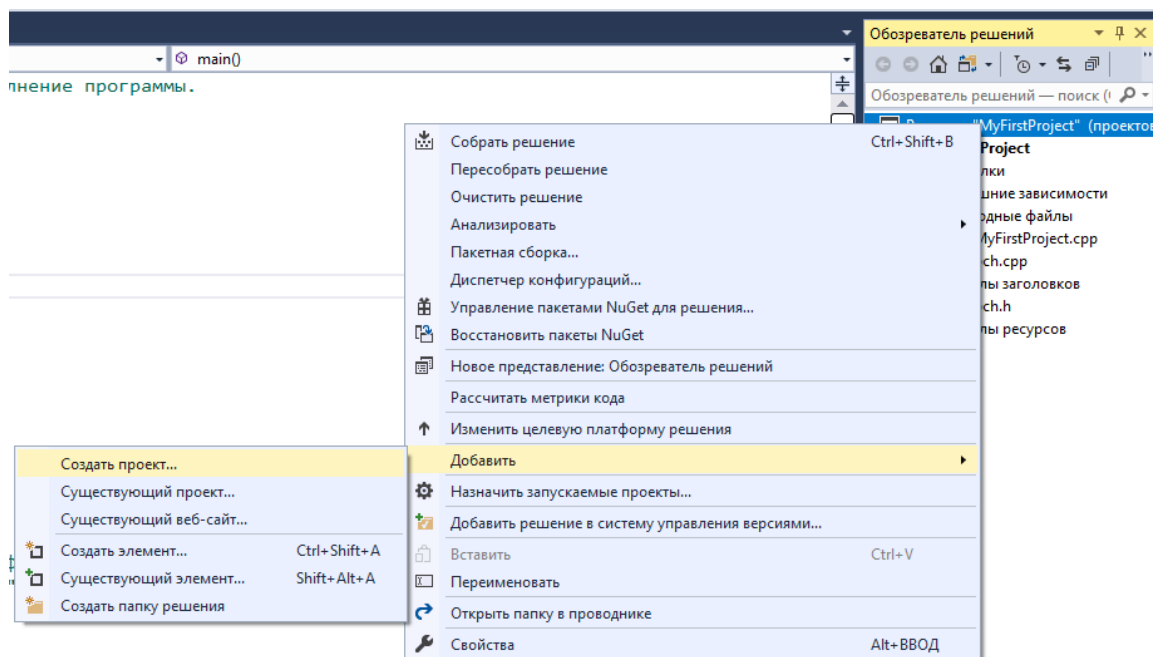


Рис. 10. Добавление нового проекта в решение

После этого появится окно для задания настроек нового проекта.

По умолчанию всегда запускается проект, которой создан первым. Чтобы это изменить, нужно в «Обозревателе решений» выполнить на имени проекта щелчок **правой кнопкой мыши** и в контекстном меню выбрать пункт **Назначить в качестве запускаемого проекта** (рис. 11).

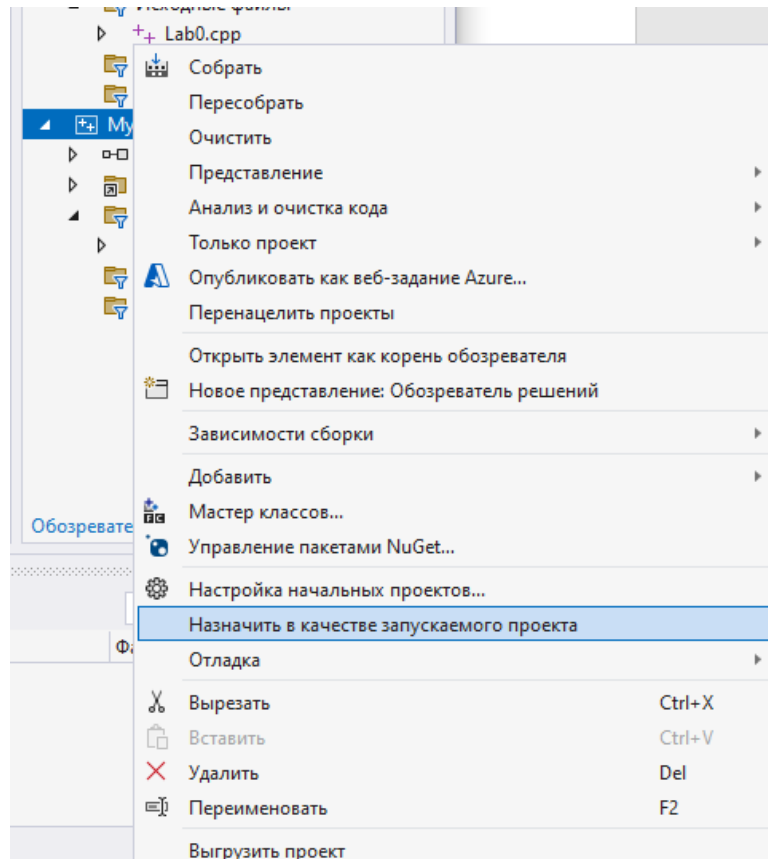


Рис. 11. Переключение между проектами

Упражнение 5. В решение **Lab0** добавьте новый проект с именем **MySecondProject**.

Скопируйте в него текст программы:

```
#include <stdio>
#include <conio.h>
int main() {
    int age;
    printf("Input your age: ");
    scanf_s("%d", &age);
    printf("Hello, human! Your age is %d.\nArtificial Intelligence is
immortal!)))", age);
    _getch();
    return 0;
}
```

Определите, что делает данная программа. Приведите пример входных данных, когда она работает некорректно. Почему?

Упражнение 6. Закомментируйте с помощью `//` строку с оператором `_getch()`. Запустите программу. Что изменилось? Какую роль выполняет этот оператор?

6. Пошаговое выполнение программы

Запустить программу на выполнение можно через **Меню – Отладка – Начать отладку**, или кнопкой  на панели инструментов (рис. 12), или нажатием **F5** на клавиатуре.

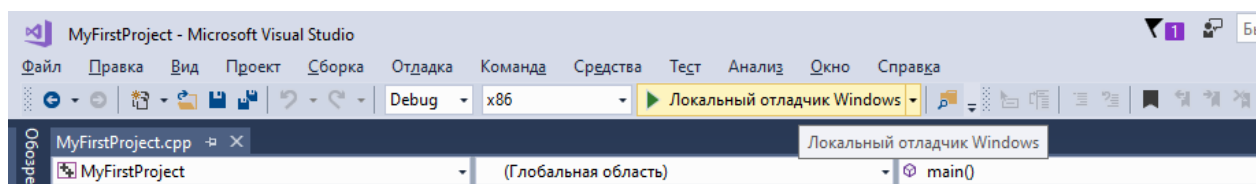


Рис. 12. Кнопка Начальный отладчик Windows

Отладка (Debugging) – это локализация и устранение ошибок. Эффективным способом отладки является пошаговое выполнение программы строка за строкой (*трассировка*).

Существует два режима пошагового выполнения программы:

- 1) трассировка с заходом в тело функции при встрече её вызова в тексте программы (**F11**, 
- 2) пошаговое выполнение функции как обычной команды без захода в ее тело (**F10**, 

Иногда нет необходимости просматривать всю программу. В этом случае используют *точки останова (BreakPoint)*.

Точка останова позволяет приостановить выполнение программы перед любой выполняемой инструкцией (оператором) для того, чтобы продолжить выполнение программы либо в пошаговом режиме, либо в непрерывном режиме до следующей точки останова.

Чтобы задать точку останова перед некоторым оператором, необходимо установить перед ними текстовый курсор и нажать клавиши **F9** или воспользоваться опцией подменю **Отладка – Создать точку останова** (рис. 12). Рядом со строкой, на которую установлено прерывание, появляется большая красная точка (рис. 13). Убрать точку останова можно повторным нажатием клавиш **F9**, при этом курсор должен находиться на строке, с которой снимается точка прерывания. Также установить и удалить точку прерывания можно кликнув левой кнопкой мыши по месту, где она должна появиться. В программе может быть несколько точек останова.

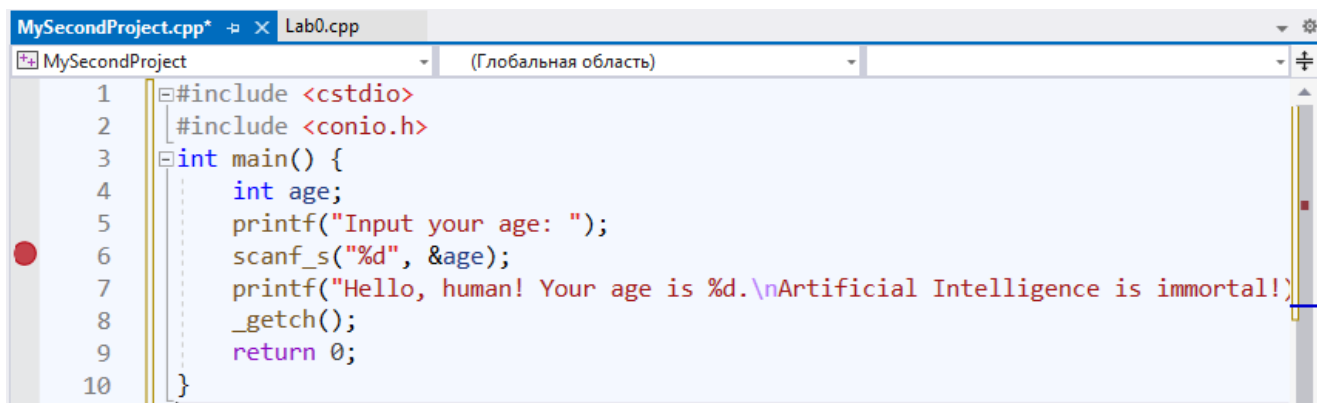


Рис. 13. Точка останова

Для наблюдения за изменением значений переменных в ходе выполнения программы используется окно **Контрольные значения** (Меню – Отладка – Окна – Контрольные значения), данное окно доступно только во время выполнения программы. Чтобы добавить переменные в это окно, необходимо во время выполнения программы выделить требуемую переменную и в контекстном меню выбрать пункт **Добавить контрольное значение** (рис. 14).

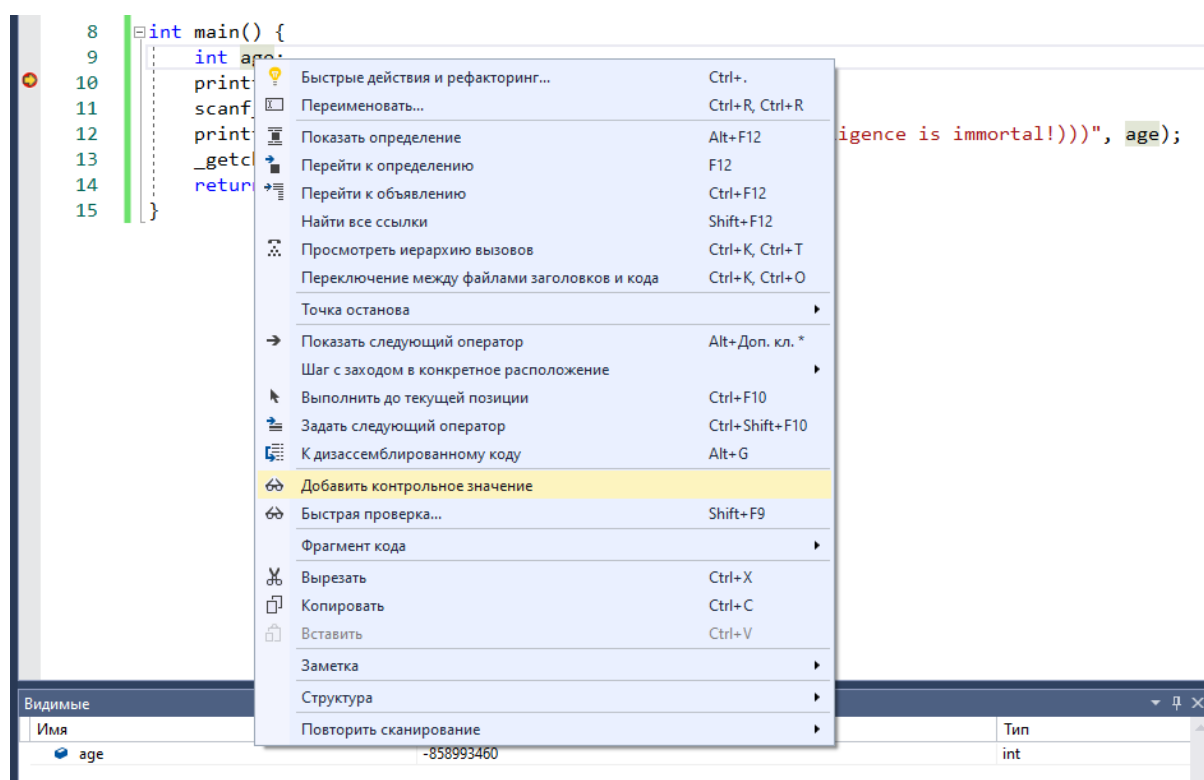


Рис. 14. Добавление переменных в окно Контрольные значения

Также контрольное значение можно добавить, вписав имя соответствующей переменной в пустую строку окна **Контрольные значения**.

Указанная переменная размещается в окне **Контрольные значения**, создаваемом в нижней части экрана, и, в процессе отладки, через это окно можно наблюдать за изменением размещенных в нем переменных. Удалить переменную из окна можно при помощи клавиши **Del**, предварительно выделив её подсветкой.

Упражнение 7. Выполните программу из упражнения 7 пошагово с просмотром промежуточных значений переменных.

Установите точку останова на строке `scanf_s("%d", &age);` и запустите программу. Что произойдет?

Посмотрите значение переменной `age`.

Продолжите выполнять программу пошагово, отслеживая изменения переменной `age`.

Упражнение 8. Внесите в программу ошибку и посмотрите сообщение в окне ошибок, после чего исправьте внесенную ошибку:

- ✓ удалите символ `;` в конце любой строки;
- ✓ добавьте в код лишний символ;
- ✓ закомментируйте строку `int age;`
- ✓ закомментируйте строку `scanf_s("%d", &age);`;
- ✓ удалите символ `&` в строке `scanf_s("%d", &age);`;
- ✓ закомментируйте строку `return 0;`
- ✓ добавьте лишнюю закрывающую фигурную скобку;
- ✓ удалите закрывающую фигурную скобку.

Задание для самостоятельного выполнения

Скопируйте на свой компьютер папку **Task4Lab1** с сетевого диска \\lib16\Students\.

Откройте в Visual Studio решение Task4Lab1.sln.

Определите, какие алгоритмы реализованы в проектах Task1 и Task2.

Напишите комментарии к существенным строкам кода.

Что произойдёт, если для программы из проекта Task1 ввести отрицательное число или действительное число, например, 3.14?

Что будет, если введённое пользователем число будет очень большим (3214569774123)?

7. Работа с онлайн-платформой для решения задач по программированию Яндекс.Контест

Платформа Яндекс.Контест поддерживает *два способа авторизации*: с помощью Яндекс ID или с логином и паролем, полученными от организаторов соревнования.

Мы будем использовать вход с помощью логина и пароля от организаторов соревнования, которыми являются ведущие преподаватели курса. Полученные логин и пароль будут действовать весь семестр, поэтому забывать или терять их нежелательно.

Упражнение 9. Войдите на платформу по адресу <https://official.contest.yandex.ru/contest/51654/enter/>, нажмите на ссылку **авторизация** (рис. 15). На открывшейся странице введите свой уникальный логин и пароль, полученный от преподавателя (рис. 16).

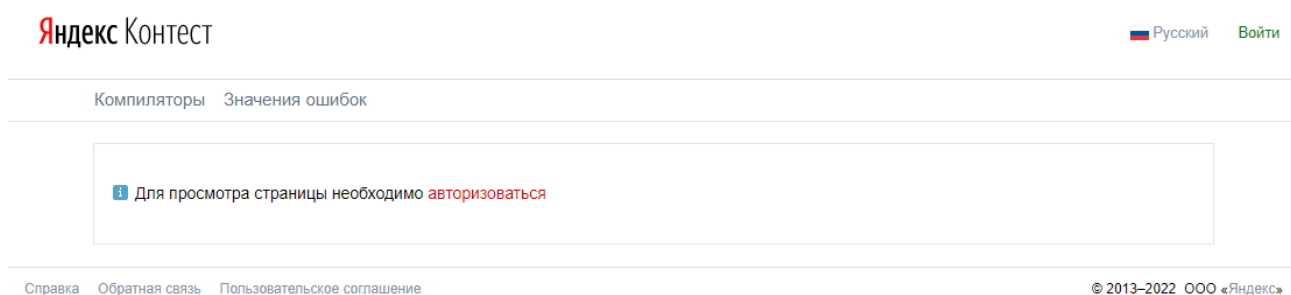


Рис. 15

Авторизация

Авторизуйтесь, используя предоставленные координатором логин и пароль

Логин

Пароль

Войти

Рис. 16. Страница авторизации

В лабораторных работах часть заданий предполагает автоматизированную проверку на платформе Яндекс.Контест. Для таких лабораторных работ будет создано отдельное соревнование с соответствующим названием.

Упражнение 10. Отправьте на проверку исходный код задачи из упражнения 3. Прочитайте внимательно Примечания к задачам.

Задание для самостоятельного выполнения

Используя примеры из упражнений, пройдите контест ОП Лабораторная работа 0.

Обратите внимание, что при автоматической проверке решений диалог с пользователем не требуется. **Соответствующие операторы нужно убрать!**

Кроме того, команду `scanf_s` нужно заменить на `scanf` из-за использования другого компилятора.

Ссылка для скачивания VS 2022:

<https://visualstudio.microsoft.com/ru/vs/community/>